

LISTA DE EXERCÍCIOS — PROGRAMAÇÃO ESTRUTURADA 26.2

1) Busca de Extremos com Aritmética de Ponteiros

Descrição: Crie uma função que percorra um vetor de inteiros utilizando exclusivamente aritmética de ponteiros. É estritamente proibido utilizar índices ou a notação de acesso por colchetes (`[]`) para percorrer o vetor. A função deve encontrar o menor e o maior valor do vetor e armazenar os resultados nas variáveis fornecidas pelos parâmetros de saída ``menor`` e ``maior``, recebidos por ponteiro.

Assinatura:

```
void encontraExtremos(int *vetor, int tamanho, int *menor, int *maior);
```

Exemplo: Entrada: [15, 3, 9, 21, 7]

Saída: min = 3, max = 21

2) Geração de Sequências de Tamanho K

Descrição: Escreva uma função recursiva que, dada uma string contendo ``n`` caracteres e um inteiro ``k``, imprima todas as sequências de comprimento ``k`` que podem ser formadas utilizando os caracteres da string. A repetição de caracteres é permitida e a ordem dos caracteres importa. Portanto, cada uma das ``k`` posições da sequência pode receber qualquer caractere da string original. Considere ``k`` ≥ 1 .

Assinatura:

```
void k_str(char *str, int k);
```

Exemplo: Entrada: str = "abc", k = 2

Saída: aa, ab, ac, ba, bb, bc, ca, cb, cc

Observação: A ordem de impressão das sequências não é considerada parte do resultado, desde que todas as sequências possíveis sejam geradas exatamente uma vez.

3) Sequência de Fibonacci (Estritamente Recursiva)

Descrição: Escreva uma função estritamente recursiva que receba um número inteiro ``n`` (onde ``n`` ≥ 0) e retorne o `n`-ésimo termo da sequência de Fibonacci, considerando $F(0) = 0$ e $F(1) = 1$. A implementação da função deve ser estritamente recursiva: não utilize ``for``, ``while``, ``do-while`` ou outra forma de iteração para calcular o resultado.

Assinatura:

```
int fibonacci(int n);
```

Exemplo: Entrada: 6

Saída: 8 (Sequência: 0, 1, 1, 2, 3, 5, 8)

4) Preenchimento de Matriz em Zigue-Zague

Descrição: Receba uma matriz quadrada vazia de dimensão ``n`` x ``n`` e preencha-a como uma matriz triangular superior. Os elementos abaixo da diagonal principal devem permanecer com valor 0. Os elementos da diagonal principal e acima dela devem ser preenchidos com uma contagem sequencial

em zigue-zague por linha.

Regra do zigue-zague: A primeira linha deve ser preenchida da esquerda para a direita; a segunda, da direita para a esquerda; a terceira, da esquerda para a direita; e assim sucessivamente. A alternância ocorre apenas entre os elementos que pertencem à região da matriz na diagonal principal ou acima dela.

Assinatura:

```
void preencheMatriz(int **matriz, int n);
```

Representação da matriz: Considere que `matriz` é uma matriz `n x n` já alocada e inicializada. A função é responsável apenas pelo preenchimento.

Exemplo: Entrada: $n = 3$

Saída:

```
1 2 3
0 5 4
0 0 6
```

5) Maior Soma entre Blocos $M \times N$

Descrição: Escreva uma função que receba as dimensões de uma matriz de inteiros (`L` linhas e `C` colunas), as dimensões dos blocos internos (`M` linhas e `N` colunas) e a própria matriz. Sabe-se que a matriz principal é dividida em blocos menores de tamanho `M x N`, sem sobreposição, sendo `L` múltiplo de `M` e `C` múltiplo de `N`. A função deve calcular e retornar a maior soma obtida entre os elementos de um único bloco `M x N`. Não é necessário procurar submatrizes arbitrárias ou sobrepostas: devem ser considerados somente os blocos definidos pela divisão da matriz.

Assinatura:

```
int maxSubmatriz(int L, int C, int M, int N, int **matriz);
```

Exemplo: Entrada: $L = 4, C = 4, M = 2, N = 2$

Matriz:

```
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15 16
```

Saída: 54 (referente ao bloco inferior direito: $11 + 12 + 15 + 16 = 54$)

6) Processamento de Comandos de Strings

Descrição: Um arquivo de texto contém uma série de instruções, uma por linha. Cada instrução é composta por um caractere de comando e uma palavra (string sem espaços), separados por um único espaço. Os comandos válidos são: `I` (inverter a palavra) e `M` (converter todas as letras minúsculas da palavra para maiúsculas). O programa deve ler o arquivo de entrada e gerar um arquivo de saída executando as instruções na mesma ordem em que aparecem.

Regras: Em cada linha, o comando aparece primeiro e a palavra aparece em seguida. Caracteres que não sejam letras minúsculas permanecem inalterados durante o comando `M`. Caso apareça um comando diferente de `I` ou `M`, a instrução deve ser ignorada e nenhuma linha correspondente deve ser escrita no arquivo de saída.

Assinatura:

```
void processaComandos(char *nomeArqEntrada, char *nomeArqSaida);
```

Exemplo:

Arquivo de entrada:

I Roma

M teste

Arquivo de saída:

amOR

TESTE

7) Palavra Mais Frequente

Descrição: Escreva um programa que leia um arquivo de texto, determine qual palavra aparece com maior frequência e imprima essa palavra junto com o número de vezes que ela apareceu.

Definição de palavra: Considere como palavra uma sequência de caracteres alfabéticos. Espaços e sinais de pontuação são considerados separadores. A comparação deve ser feita sem diferenciar letras maiúsculas de minúsculas; por exemplo, `Casa`, `casa` e `CASA` devem ser consideradas a mesma palavra.

Empate: Caso duas ou mais palavras tenham a mesma frequência máxima, imprima a palavra que aparecer primeiro no arquivo entre as palavras empatadas.

Assinatura:

```
void contaPalavraMaisFrequente(char *nomeArquivo);
```

Exemplo: Arquivo de entrada contendo a frase: "ponteiro matriz ponteiro string"

Saída: A palavra "ponteiro" apareceu 2 vezes.